

Study Guide: CS 173 – Discrete Structures

Generated with NotebookLM. May contain inaccuracies.

This study guide covers key concepts in discrete structures and theoretical computer science, drawing from course materials on logic, proofs, number theory, sets, relations, functions, graphs, trees, algorithms, complexity (Big-O, NP), state diagrams, and countability.

Based on notes by Ben Pegg

Contents

I. Core Mathematical Prerequisites & Foundations	3
A. Algebraic Manipulations & Numerical Functions	3
B. Numbers & Sets	3
C. Logic: Propositional & Predicate	3
D. Number Theory	4
II. Proof Techniques	4
A. General Proof Approaches	4
B. Specific Proof Methods	4
C. Induction	5
III. Discrete Structures & Advanced Topics	5
A. Functions	5
B. Relations	6
C. Graphs	6
D. Trees	7
E. Algorithms & Complexity	8
IV. Countability & Uncomputability	9
A. Cardinality	9
B. Countable Sets	9
C. Uncountable Sets	10
D. Uncomputability	10
V. Problem Solving & Style	10
Quiz	11
Quiz Answer Key	12
Essay Format Questions (No Answers)	13
Glossary of Key Terms	14

I. Core Mathematical Prerequisites & Foundations

A. Algebraic Manipulations & Numerical Functions

- Equations & Inequalities: Basic operations, solving for variables.
- Fractions & Absolute Value: Understanding and manipulating these concepts.
- Squares & Square Roots: Basic properties.
- Complex Numbers: Familiarity with 'i' (square root of -1).
- Polynomials: Solving, factoring, and finding roots for 2nd order polynomials (easy cases).
- Exponents & Logarithms: Basic rules for manipulation.
- Defining & Composing Numerical Functions: E.g., $g(f(x))$.

B. Numbers & Sets

- Standard Number Sets: $\mathbb{R}$ (reals), $\mathbb{N}$ (naturals, includes 0), $\mathbb{Z}$ (integers), $\mathbb{Z}^+$ (positive integers), $\mathbb{Q}$ (rationals), $\mathbb{C}$ (complex numbers).
- Set Membership Notation: $x \in \mathbb{Z}$.
- Interval Notation: $[a,b]$ (closed), (a,b) (open) for real numbers.
- Floor & Ceiling Functions: $\lceil x \rceil$ and $\lfloor x \rfloor$.

C. Logic: Propositional & Predicate

- Basic Logical Operators: Truth tables (especially implication), understanding inclusive "or".
- Quantifiers: Universal ($\forall$), Existential ($\exists$), shorthand, and scope.
- Translation: Between English and logical shorthand.
- Logical Equivalences: Distributive, commutative, associative laws; $p \rightarrow q \equiv (\neg p) \vee q$.
- Truth Tables & Counter-examples: Using them to verify logical equivalences.
- Statements: Identifying non-statements, and statements with unbound variables.
- Truth Evaluation: Determining the truth of complex statements given basic truths.
- If/Then Statements: Identifying hypothesis and conclusion.
- Negation & Contrapositive: Forming and simplifying them; understanding their equivalence.
De Morgan's laws for quantifiers and propositions.
- Binding & Scope: Understanding how quantifiers bind variables and their scope.

D. Number Theory

- Definitions: Divides, multiple, odd, even, prime, composite, perfect square, rational number, relatively prime.
- Special Cases: Applying definitions to zero and negative numbers (e.g., zero is even).
- Prime Factorization: Factoring numbers into primes (≥ 2); 0 and 1 are not prime.
- GCD & LCM: Definitions and computation for small values.
- Fundamental Theorem of Arithmetic: Statement and understanding of unique prime factorization.
- Irrationality of $\sqrt{2}$: Knowing it's irrational.
- Infinitude of Primes: Knowing this fact.
- Congruence Modulo k : Definition and properties.
- Division Algorithm Theorem: Statement.
- Euclidean Algorithm: Computation of GCD for larger numbers, tracing execution.
- Proofs: Direct proofs for simple number theory claims (e.g., divisibility properties).

II. Proof Techniques

A. General Proof Approaches

- Direct Proof: Starting with given information, proceeding logically to the conclusion.
- Disproof: Using concrete counter-examples for universal claims.
- Existential Claims: Proving by providing specific values.
- Universal Claims: Proving by working with a representative object (not a concrete example).
- If/Then Statements: Assuming hypothesis, proving conclusion.

B. Specific Proof Methods

- Proof by Cases: Dividing the proof into multiple scenarios based on conditions.
- Proof by Contrapositive: Converting $P \rightarrow Q$ to $\neg Q \rightarrow \neg P$ and proving the latter directly.
- Proof by Contradiction: Assuming the negation of the claim and deriving a contradiction.
- Two-way Bounding: Proving an equality by establishing upper and lower bounds (e.g., set equality by proving inclusion in both directions, chromatic number justification).

- Without Loss of Generality (WLOG): Simplifying proofs by making an additional assumption that doesn't restrict the generality of the proof.

C. Induction

- Mathematical Induction: Proving $P(n)$ for all natural numbers n .
- Outline: Base case, inductive hypothesis, inductive step.
- Strong vs. Weak Induction: Understanding when a strong inductive hypothesis (assuming $P(i)$ for all $i < k$) is necessary.
- Applications: Summation formulas, divisibility, geometric problems, properties of recursively defined sequences/trees.
- Recursion Trees: Analyzing recursive definitions by summing work at each level of the tree.

III. Discrete Structures & Advanced Topics

A. Functions

- Notation: $f:A \rightarrow B$, domain, co-domain, image (of function), pre-image (of value).
- Composition: Definition and computation ($g \circ f$).
- Identity Function: id_A .
- Non-functions: Identifying relations that are not functions (e.g., multiple outputs for one input).
- Properties: One-to-one (Injective): Definition ($f(x)=f(y) \rightarrow x=y$), identifying, proving.
- Onto (Surjective): Definition ($\forall b \in B, \exists a \in A, f(a)=b$), identifying, proving.
- Bijective: Both one-to-one and onto.
- Increasing/Strictly Increasing: Definitions and proofs (e.g., strictly increasing implies one-to-one).
- Pigeonhole Principle: Application in counting and proofs (if n objects into k bins with $n > k$, at least two objects share a bin).
- Permutations & Combinations: Counting ordered/unordered selections.

B. Relations

- Definition: Subset of $A \times A$. xRy .

- Properties: Reflexive: xRx for all x .
- Irreflexive: $x \not R x$ for all x .
- Symmetric: $xRy \rightarrow yRx$.
- Antisymmetric: $(xRy \text{ and } yRx) \rightarrow x=y$.
- Transitive: $(aRb \text{ and } bRc) \rightarrow aRc$.
- Types of Relations: Equivalence Relation: Reflexive, symmetric, transitive.
- Partial Order: Reflexive, antisymmetric, transitive.
- Strict Partial Order: Irreflexive, antisymmetric, transitive.
- Equivalence Classes: Understanding and defining $[x]R$.

C. Graphs

- Basics: Vertices (nodes), edges, simple graph (no loops/multi-edges).
- Naming Edges: vw or $\{v,w\}$.
- Walks: Edge/node sequences, open/closed walks, length.
- Terminology: Adjacent/neighboring, endpoints, incident, degree of vertex, subgraph.
- Special Graphs: K_n (complete), C_n (cycle), W_n (wheel), $K_{n,m}$ (complete bipartite). Knowing shorthand, structure, vertices, edges.
- Paths: Walks with no repeated nodes.
- Cycles: Closed walks with at least three nodes and no repeated nodes (except start/end).
- Connectivity: Connected graph, connected component, cut edge, acyclic.
- Distances: Length of path, distance between nodes, diameter.
- Euler Circuit: Definition, condition for existence (all vertices even degree).
- Handshaking Theorem: Sum of degrees equals twice number of edges.
- Bipartite Graph: Definition.
- Properties & Analysis: Finding connected components.
- Determining Euler circuits.
- Counting paths between nodes.
- Graph Isomorphism: Definition (bijection preserving edges).

- Proving isomorphism (providing bijection).
- Counting isomorphisms.
- Proving non-isomorphism (finding differing invariant features: node/edge count, degree sequence, small subgraphs).
- Graph Coloring: Vertex coloring definition (adjacent vertices different colors).
- Chromatic number $\chi(G)$.
- Applications (register allocation).
- Relationship to maximum degree (colorable with $D+1$ colors).
- Greedy coloring algorithm.
- Planar Graphs: Definition (drawable without edge crossings).
- Faces: Regions divided by a planar drawing, degree of a face, Handshaking theorem for faces.
- Euler's Formula: $v - e + f = 2$ for connected planar graphs.
- Corollaries: $e \leq 3v - 6$ (for $v \geq 3$), existence of a node with degree < 6 .
- Kuratowski's Theorem: A graph is planar IFF it does not contain a subgraph that is a subdivision of K_5 or $K_{3,3}$.
- Four Color Theorem: Planar graphs are 4-colorable.
- Platonic Solids: Connection to planar graphs and constraints on faces/degrees.

D. Trees

- Recursive Definition: Binary tree (single node, or root with children).
- Terminology: Root, leaf, internal node, parent, child, sibling, ancestor, descendent, level, height.
- Types: Binary, n -ary, balanced. (Definitions for full/complete provided).
- Key Facts: nodes = edges + 1.
- Full m -ary tree: $mi+1$ total nodes if i internal nodes.
- Full binary tree: $n+1$ leaves if n internal nodes (total $2n+1$ nodes).
- Binary tree of height h : leaves between 1 and 2^h , nodes between $(h+1)$ and $(2^{h+1} - 1)$.
- Height of full complete binary tree with n nodes/leaves is $\log(n)$.

- Context-Free Grammars (Grammar Trees): Variables, rules, start symbols, terminal symbols.
- Interpreting grammar rules (ϵ , $|$).
- Determining if a tree/string can be generated.
- Building trees for specific strings.
- Recursion Trees: Visualizing and summing work for recursive algorithms (e.g., for $T(n) = 2T(n/2) + n$).
- Tree Induction: Proving properties of trees using induction, often on height.

E. Algorithms & Complexity

- Big-O Notation: Purpose: Asymptotic analysis of running time for large inputs, ignoring multiplicative constants.
- Formal Definition: $f(n) = O(g(n))$ if $\exists c, k > 0$ such that $0 \leq f(n) \leq cg(n)$ for $n \geq k$.
- Asymptotic Relationships: $\approx$ (similar), $\ll$ (smaller).
- Ordering of Primitive Functions: $1 \ll \log n \ll n \ll n \log n \ll n^2 \ll n^3 \ll \dots \ll 2^n \ll 3^n \ll n!$.
- Dominant Term Method: For sums, only the fastest-growing term matters.
- Proving Big-O: Finding suitable c and k .
- Theta (Θ) Notation: $f(n) = \Theta(g(n))$ if $f(n) = O(g(n))$ and $g(n) = O(f(n))$.
- Logarithm Base: Does not affect Big-O.
- Basic Data Structures: Arrays: $O(1)$ access, $O(n)$ for length change/middle insert/delete.
- Linked Lists: $O(n)$ access (except head), $O(1)$ for head insert/remove.
- Algorithm Analysis Methods: Nested loops, resource consumption, recursive definitions.
- Specific Algorithms & Their Big-O Running Times: Closest Pair (Nested Loops): $O(n^2)$.
- Graph Connected Component (BFS/DFS variant): $O(n+m)$ (n nodes, m edges).
- Binary Search: $O(\log n)$.
- Merge Two Sorted Lists: $O(n)$.
- Merge Sort: $O(n \log n)$.
- Towers of Hanoi: $O(2^n)$.

- Karatsuba's Algorithm (Integer Multiplication): $O(n^{1.585})$. (Naive is $O(n^2)$).
- Analyzing loops: Expressing running time with summations.
- Analyzing recursive algorithms: Expressing running time as recursive definitions.
- Complexity Classes: EXP: Problems requiring exponential time (e.g., Towers of Hanoi).
- NP (Non-deterministic Polynomial time): Problems where "yes" answers can be quickly (polynomial time) justified/verified. Solution might be exponential.
- Co-NP: Problems where "no" answers can be quickly (polynomial time) justified/verified.
- P (Polynomial time): Problems solvable in polynomial time. $P \subseteq NP$ and $P \subseteq co-NP$. (P vs NP is an open question).
- NP-Complete: Problems in NP for which a polynomial-time algorithm would imply all problems in NP can be solved in polynomial time. (e.g., graph colorability ($k \geq 3$), circuit satisfiability, propositional logic satisfiability, traveling salesman).
- Example: Graph 2-colorability (bipartite) is solvable in polynomial time.

IV. Countability & Uncomputability

A. Cardinality

- Definition: Two sets A and B have the same cardinality ($|A| = |B|$) iff there is a bijection from A to B.
- $|A| \leq |B|$: There is a one-to-one function from A to B.
- Cantor-Schroeder-Bernstein Theorem: If $|A| \leq |B|$ and $|B| \leq |A|$, then $|A| = |B|$.

B. Countable Sets

- Countably Infinite: A set with the same cardinality as the natural numbers (N) or integers (Z).
- Countable: Either finite or countably infinite.
- Examples of Countable Sets: Integers, natural numbers.
- Pairs of integers or natural numbers (N^2).
- Rational numbers (Q).
- Finite-length strings over a finite character set.
- All finite formulas or computer programs.

- Subsets of countable sets.
- Finite product of countable sets.
- Countable union of countable sets.

C. Uncountable Sets

- Definition: Infinite sets too large to be countable.
- Examples of Uncountable Sets: Real numbers ($\mathbb{R}$), intervals of reals (e.g., $[0,1]$).
- Power set of the integers ($P(\mathbb{Z})$ or $P(\mathbb{N})$).
- Products containing an uncountable set (e.g., pairs of reals).
- Sets containing an uncountable subset.
- Infinite-length sequences of bits.
- All functions from an infinite set (e.g., integers) to a set with at least two elements.
- Key Facts: $|P(A)| > |A|$ for any set A .
- $|\mathbb{R}^2| = |\mathbb{R}|$ (shown by interleaving digits).

D. Uncomputability

- Functions vs. Formulas/Programs: Countable number of formulas/programs vs. uncountable number of functions implies existence of uncomputable functions.
- Halting Problem: Impossible to build a program that determines if any arbitrary input program halts or runs forever.
- Program Behavior: Halts (finite trace), loops (repeating states), runs forever (infinite states without repeating).
- Connection to Tilings: Aperiodic tilings relate to uncomputability and non-periodic behavior.

V. Problem Solving & Style

- Reading Mathematical Definitions: Understanding "if" vs. "iff", careful use of variables.
- Proof Structure: Clear variable declarations, stating hypotheses and conclusions, logical order.
- Using Pseudocode: Understanding its abstraction level.
- LaTeX: Mentioned as a tool for writing mathematical proofs.

- Rubrics: Understanding criteria for evaluating proofs (variable declaration, set definitions, algebraic details, proof outline, style/clarity, base cases, IH, case structure, applying IH, recursion details, explanation for Big-O).

Quiz

Instructions: Answer each question in 2-3 sentences.

1. Explain the key difference between a "simple graph" and a general graph. Why is this distinction important in graph theory?
2. Define what it means for a function $f: A \rightarrow B$ to be "onto". Provide an example of a function that is not onto from $\mathbb{Z}$ to $\mathbb{Z}$.
3. What is the purpose of Big-O notation in algorithm analysis? What two main aspects of a program's running time does it intentionally ignore?
4. State the "Handshaking Theorem" for graphs. What is its implication regarding the number of odd-degree vertices in any graph?
5. Distinguish between a "countable" set and an "uncountable" set. Give one example of each.
6. Describe the core idea behind a "proof by contradiction". When is this proof technique typically most effective?
7. Explain the "Fundamental Theorem of Arithmetic". Why is it significant for number theory and related computer science applications?
8. How does the "Pigeonhole Principle" work? Provide a simple, non-mathematical example to illustrate it.
9. What is an "NP-complete" problem? Why is it a significant concept in computational complexity theory?
10. Define a "context-free grammar" and describe its main components. What is its primary use in theoretical computer science?

Quiz Answer Key

1. A "simple graph" is an undirected graph that contains neither "loops" (edges connecting a vertex to itself) nor "multi-edges" (multiple edges connecting the same pair of vertices). This

distinction simplifies many graph theory problems and definitions by removing common ambiguities and edge cases. Most standard graph theorems and algorithms assume simple graphs unless otherwise specified.

2. A function $f: A \rightarrow B$ is "onto" (or surjective) if its image is equal to its entire co-domain B . This means that for every element y in B , there exists at least one element x in A such that $f(x) = y$. An example of a function $f: \mathbb{Z} \rightarrow \mathbb{Z}$ that is not onto is $f(x) = 2x$, because odd integers in the co-domain (like 1 or 3) have no corresponding integer pre-image.
3. Big-O notation is used for the asymptotic analysis of an algorithm's running time, focusing on its performance for very large inputs. It intentionally ignores both the behavior on small inputs (where constant factors might dominate) and multiplicative constants in the running time function. This allows for a portable and durable analysis across different implementations and hardware platforms.
4. The Handshaking Theorem states that the sum of the degrees of all vertices in any undirected graph is equal to twice the number of edges. Its implication is that the number of vertices with an odd degree in any graph must always be even. This is because summing an even number of odd numbers will result in an even total, and the total sum of degrees must be even.
5. A "countable" set is one that is either finite or "countably infinite," meaning it has the same cardinality as the natural numbers (a bijection exists between it and $\mathbb{N}$ or $\mathbb{Z}$). An "uncountable" set is an infinite set that is "too large" to be countable, meaning no such bijection exists. An example of a countable set is the set of integers ($\mathbb{Z}$), while an example of an uncountable set is the set of real numbers ($\mathbb{R}$).
6. Proof by contradiction involves showing that a claim P is true by demonstrating that its negation ($\neg P$) leads to a logical inconsistency or a statement known to be false. If assuming $\neg P$ results in a contradiction, then $\neg P$ cannot be true, thus P must be true. This technique is often most effective for proving non-existence claims or claims about irrationality (e.g., $\sqrt{2}$ is irrational).

7. The Fundamental Theorem of Arithmetic states that every integer greater than or equal to 2 can be written as a product of one or more prime factors, and this prime factorization is unique except for the order in which the factors are written. It's significant because it establishes a foundational property of integers, providing a unique "fingerprint" for each number, which is crucial for concepts like GCD, LCM, and modern cryptographic algorithms.
8. The Pigeonhole Principle states that if you have more "objects" than "bins" into which these objects are placed, then at least one bin must contain more than one object. For example, if you have 13 socks and only two colors (black and white), at least one color must have at least 7 socks, because if both had 6 or fewer, you'd only have 12 or fewer socks total.
9. An NP-complete problem is a problem that belongs to the complexity class NP, and for which a polynomial-time algorithm would imply that all problems in NP can be solved in polynomial time. It is a significant concept because it represents the "hardest" problems in NP; if a fast solution to one NP-complete problem is found, it effectively provides a fast solution for a vast class of other important computational problems.
10. A context-free grammar is a set of rules that specify the hierarchical structure of sequences of "terminal symbols" (like words or characters). Its main components are "variables" (non-terminal symbols that denote structural categories), "rules" (production rules showing how variables can be replaced by sequences of other variables and terminals), "start symbols" (the root of a parse tree), and "terminal symbols" (the leaves of the parse tree). Its primary use is in defining the syntax of programming languages and analyzing the structure of natural language sentences.

Essay Format Questions (No Answers)

1. Compare and contrast the proof techniques of "proof by contrapositive" and "proof by contradiction." Discuss situations where each is particularly well-suited, and explain why one might be preferred over the other in certain contexts. Use examples from the course material to illustrate your points.

2. Explain the concept of "countability" in set theory, detailing the distinction between finite, countably infinite, and uncountable sets. Discuss how bijections and one-to-one functions are used to establish cardinality. Provide arguments for why the set of rational numbers is countable while the set of real numbers is uncountable, referring to relevant theorems or proof techniques.
3. Analyze the significance of "NP-complete" problems in theoretical computer science. Define NP, co-NP, and P, and discuss their relationships. Provide specific examples of NP-complete problems mentioned in the course material and explain why the question of whether $P=NP$ is considered one of the most important unsolved problems in computer science.
4. Describe how Big-O notation provides a valuable framework for analyzing algorithm efficiency. Discuss the process of determining the Big-O running time for algorithms, illustrating with examples of different complexity classes (e.g., $O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$, $O(n^2)$, $O(2^n)$). Explain why multiplicative constants and behavior on small inputs are ignored in this analysis.
5. Discuss the role of different types of graphs and their properties in modeling real-world problems. Choose three distinct graph types or properties (e.g., bipartite graphs, planar graphs, Euler circuits, graph isomorphism) from the course material and for each, describe its definition, key characteristics, and a specific application or type of problem it can effectively model.

Glossary of Key Terms

- Acyclic Graph: A graph that contains no cycles.
- Adjacent Vertices: Two vertices connected by an edge. Also called neighbors.
- Algorithm: A finite sequence of unambiguous instructions for solving a problem.
- Antisymmetric Relation: A relation R on a set A where for all distinct $x, y \in A$, if xRy then $y \not R x$. Equivalently, if xRy and yRx , then $x = y$.
- Asymptotic Analysis: The study of the behavior of functions as their input approaches infinity, primarily used to evaluate algorithm efficiency.

- Asymptotically Similar ($f(n) \approx g(n)$): $\lim_{n \rightarrow \infty} f(n)/g(n) = c$, where c is a real number.
- Asymptotically Smaller ($f(n) \ll g(n)$): $\lim_{n \rightarrow \infty} f(n)/g(n) = 0$.
- Base Case (Induction/Recursion): The initial condition(s) or smallest instances of a problem for which a claim or recursive definition is directly given, stopping the recursion.
- Big-O Notation ($O(g(n))$): A mathematical notation that describes the upper bound of the growth rate of a function's running time or space requirements as the input size grows. $f(n) = O(g(n))$ if there exist positive constants c and k such that $0 \leq f(n) \leq c \cdot g(n)$ for all $n \geq k$.
- Bijection (Bijective Function): A function that is both one-to-one (injective) and onto (surjective). It maps each element of its domain to exactly one element of its co-domain, and every element in the co-domain has exactly one pre-image.
- Binary Tree: An m -ary tree where each node has at most two children.
- Bipartite Graph: A graph whose vertices can be divided into two disjoint sets, V_1 and V_2 , such that every edge connects a vertex in V_1 to one in V_2 (no edges within V_1 or V_2).
- Bit String: A sequence composed solely of binary digits (0s and 1s).
- Cardinality: The number of distinct elements in a set. For infinite sets, it compares their "size" based on the existence of bijections.
- Cartesian Product ($A \times B$): The set of all possible ordered pairs where the first element is from set A and the second is from set B .
- Chromatic Number ($\chi(G)$): The minimum number of colors required to color a graph G such that no two adjacent vertices share the same color.
- Circuit Satisfiability (Circuit SAT): An NP-complete problem asking if there is an input assignment that makes a given Boolean circuit output "true".
- Closed Walk: A walk in a graph where the starting and ending vertices are the same.
- Co-domain: The set of all possible output values of a function.
- Co-NP: The set of decision problems for which "no" answers can be verified in polynomial time.

- Combination (k-combination): An unordered selection of k items from a larger set of n distinct items, without replacement. The number of k -combinations is given by $C(n, k)$ or $(n \text{ choose } k)$.
- Complete Bipartite Graph ($K_{m,n}$): A bipartite graph with m vertices in one set and n vertices in the other, where every vertex in the first set is connected to every vertex in the second set.
- Complete Graph (K_n): A graph with n vertices where every vertex is connected to every other vertex by a single edge.
- Composite Number: An integer $q \geq 2$ that is not prime; it has positive factors other than 1 and itself.
- Composition of Functions ($g \circ f$): A new function formed by applying one function to the results of another. $(g \circ f)(x) = g(f(x))$.
- Connected Component: A maximal connected subgraph of a graph.
- Connected Graph: A graph where there is a path (or walk) between every pair of its vertices.
- Context-Free Grammar: A formal grammar used to generate all possible strings in a given formal language. It consists of variables, rules (productions), start symbols, and terminal symbols.
- Contradiction: A statement or set of statements that are logically inconsistent or inherently false.
- Contrapositive: For an implication $P \rightarrow Q$, its contrapositive is $\neg Q \rightarrow \neg P$. It is logically equivalent to the original implication.
- Countable Set: A set that is either finite or countably infinite.
- Countably Infinite Set: An infinite set that has the same cardinality as the set of natural numbers ($\mathbb{N}$) or integers ($\mathbb{Z}$), meaning a bijection can be constructed between them.
- Cut Edge: An edge in a graph whose removal increases the number of connected components.
- Cycle Graph (C_n): A graph with n vertices forming a single cycle.
- Cycle (in graph): A closed walk with at least three distinct vertices where no vertex (except the start/end vertex) is repeated.

- Degree of a Vertex ($\deg(v)$): The number of edges incident to a vertex v .
- Diagonalization: A proof technique, often attributed to Georg Cantor, used to demonstrate that one infinite set is "larger" than another by constructing an element that cannot be in an assumed enumeration.
- Diameter of a Graph: The maximum distance between any pair of vertices in a connected graph.
- Direct Proof: A proof that starts with the given premises and logically progresses step-by-step to the desired conclusion.
- Disjoint Sets: Two sets whose intersection is the empty set (they have no elements in common).
- Divides ($a \mid b$): An integer a divides an integer b if $b = a \cdot n$ for some integer n . a is a factor or divisor of b , and b is a multiple of a .
- Domain: The set of all possible input values for a function.
- Dominant Term: In an expression for algorithm running time, the term that grows fastest as the input size increases, determining the overall asymptotic behavior.
- Euclidean Algorithm: An efficient method for computing the greatest common divisor (GCD) of two integers.
- Euler Circuit: A closed walk in a graph that traverses every edge exactly once.
- Existential Quantifier ($\exists$): "There exists" or "for some". Denotes that at least one element in a set satisfies a given property.
- EXP: The complexity class of problems solvable in exponential time.
- Face (Planar Graph): A region of the plane bounded by edges of a planar graph drawn without crossings. The unbounded outer region is also counted as a face.
- Fibonacci Numbers: A sequence $F_0=0, F_1=1, F_i = F_{i-1} + F_{i-2}$ for $i \geq 2$.
- Function: A rule that assigns to each element in a set (the domain) exactly one element in another set (the co-domain).
- Fundamental Theorem of Arithmetic: Every integer greater than or equal to 2 can be uniquely factored into a product of prime numbers (ignoring the order of factors).

- Grammar Tree (Parse Tree): A tree structure representing the syntactic structure of a string according to a formal grammar, with terminal symbols at the leaves.
- Graph: A mathematical structure consisting of a set of vertices (nodes) and a set of edges connecting pairs of vertices.
- Graph Coloring (Vertex Coloring): Assigning a "color" to each vertex of a graph such that no two adjacent vertices have the same color.
- Graph Isomorphism: A bijection between the vertex sets of two graphs that preserves adjacency, meaning the graphs have the same structural properties.
- Greatest Common Divisor (GCD): The largest positive integer that divides two or more integers without leaving a remainder.
- Halting Problem: The undecidable problem of determining whether an arbitrary computer program will finish running or continue to run forever.
- Height (Tree): The length of the longest path from the root to a leaf node.
- Image (of a Function): The set of all actual output values produced by a function when applied to all elements of its domain.
- Implication ($p \rightarrow q$): "If p , then q ." A logical connective that is false only when the hypothesis (p) is true and the conclusion (q) is false.
- Inclusion-Exclusion Principle: A counting technique to find the number of elements in the union of multiple sets, correcting for elements counted multiple times in intersections.
- Increasing Function: For a function $f: A \rightarrow B$ where A and B are sets of real numbers, f is increasing if $x \leq y$ implies $f(x) \leq f(y)$.
- Inductive Hypothesis: The assumption made in an inductive proof that the property $P(n)$ holds for n up to $k-1$ (weak induction) or k (strong induction).
- Inductive Step: The part of an inductive proof where one uses the inductive hypothesis to show that $P(k)$ (or $P(k+1)$) is true.
- Injective Function (One-to-one): A function where every distinct element in the domain maps to a distinct element in the co-domain; no two inputs map to the same output.

- Internal Node (Tree): A node in a tree that has at least one child.
- Irreflexive Relation: A relation R on a set A where $x \not R x$ for all $x \in A$. No element is related to itself.
- Kuratowski's Theorem: States that a graph is planar if and only if it does not contain a subgraph that is a subdivision of K_5 or $K_{3,3}$.
- Leaf Node (Tree): A node in a tree that has no children.
- Least Common Multiple (LCM): The smallest positive integer that is a multiple of two or more given integers.
- Linked List: A linear data structure where elements are stored in nodes, and each node points to the next node in the sequence.
- Linear Order (Total Order): A partial order where every pair of elements is comparable (either $x R y$ or $y R x$).
- Logic (Propositional/Predicate): Systems for formalizing reasoning. Propositional logic deals with propositions, while predicate logic extends this with variables and quantifiers.
- Loop (Graph): An edge that connects a vertex to itself.
- Lower Bound: A value that is less than or equal to all values in a set. In optimization or proofs, a value that is known to be the minimum possible for a quantity.
- Merge Sort: A sorting algorithm that divides a list into two halves, recursively sorts them, and then merges the sorted halves. Has $O(n \log n)$ running time.
- Multi-edge (Graph): Multiple edges connecting the same pair of vertices.
- NP: The complexity class of decision problems for which a given "yes" solution can be verified in polynomial time.
- NP-Complete (NPC): A problem in NP such that every other problem in NP can be transformed into it in polynomial time. If an NP-complete problem can be solved in polynomial time, then all problems in NP can.
- One-to-one Correspondence: Synonym for a bijection.

- **Onto Function (Surjective):** A function where every element in the co-domain is mapped to by at least one element from the domain; the image equals the co-domain.
- **Parent (Tree):** The node directly above a given node in a tree, connected by an edge.
- **Partial Order:** A binary relation that is reflexive, antisymmetric, and transitive.
- **Partition (of a Set):** A collection of non-empty subsets of a given set such that every element of the original set is in exactly one of the subsets.
- **Path (in graph):** A walk in a graph where no vertex is repeated.
- **Permutation:** An ordered arrangement of a set of objects.
- **Pigeonhole Principle:** If n items are put into k containers, with $n > k$, then at least one container must contain more than one item.
- **Planar Graph:** A graph that can be drawn in a plane without any edges crossing each other.
- **Platonic Solids:** Five regular convex polyhedra (tetrahedron, cube, octahedron, dodecahedron, icosahedron) whose faces are all congruent regular polygons and where the same number of faces meet at each vertex.
- **Power Set ($P(A)$):** The set of all possible subsets of a given set A .
- **Predicate:** A statement containing one or more variables that becomes true or false when values are substituted for the variables.
- **Pre-image:** For a function $f(x)=y$, x is a pre-image of y . An output y may have multiple pre-images.
- **Prime Number:** An integer $q \geq 2$ whose only positive factors are 1 and q .
- **Proof by Cases:** A proof technique where the argument is divided into several exhaustive cases, and each case is proved separately.
- **Proof by Contradiction:** A proof technique where one assumes the negation of the statement to be proved, and then shows that this assumption leads to a logical contradiction, thereby proving the original statement.
- **Pseudocode:** An informal high-level description of an algorithm's operating principle, often using a mix of natural language and programming constructs.

- Quantifier: A logical operator that specifies the number or quantity of elements in a domain that satisfy a property (e.g., "for all", "there exists").
- Rational Number (Q): A real number that can be expressed as a fraction m/n where m and n are integers and $n \neq 0$.
- Recursion Tree: A diagram that visually represents the work done at each level of a recursive algorithm's execution, useful for analyzing its running time.
- Recursive Definition (Recurrence Relation): A definition where an object (e.g., a function, a sequence, a set) is defined in terms of smaller instances of itself, along with one or more base cases.
- Reflexive Relation: A relation R on a set A where xRx for all $x \in A$. Every element is related to itself.
- Regular Expression: A sequence of characters that specifies a search pattern. (Briefly mentioned in context of string notation).
- Relatively Prime: Two integers a and b are relatively prime if their greatest common divisor (GCD) is 1.
- Root (Tree): The unique node in a tree that has no parent.
- Scope (Quantifier): The portion of a logical expression where a variable bound by a quantifier is defined and can be used.
- Sibling (Tree): Two nodes that share the same parent.
- Simple Graph: An undirected graph without loops or multiple edges.
- Strictly Increasing Function: For a function $f: A \rightarrow B$ where A and B are sets of real numbers, f is strictly increasing if $x < y$ implies $f(x) < f(y)$.
- Subgraph: A graph formed from a subset of vertices and a subset of edges of a larger graph.
- Subdivision (of a Graph): A graph G is a subdivision of graph H if G can be obtained from H by adding new degree-2 vertices on one or more edges of H .
- Symmetric Relation: A relation R on a set A where for all $x, y \in A$, if xRy , then yRx .

- **Terminal Symbol (Grammar):** A symbol in a formal grammar that appears in the final generated strings and cannot be further expanded.
- **Theta Notation ($\Theta(g(n))$):** A notation that describes a tight bound on the growth rate of a function. $f(n) = \Theta(g(n))$ if $f(n) = O(g(n))$ and $g(n) = O(f(n))$.
- **Towers of Hanoi:** A mathematical puzzle with a recursive solution that exhibits exponential time complexity.
- **Trace (Algorithm):** A step-by-step description of an algorithm's execution, showing the values of key variables.
- **Transitive Relation:** A relation R on a set A where for all $a, b, c \in A$, if aRb and bRc , then aRc .
- **Transition Function (δ):** In state diagrams, a function that takes a current state and an action as input and returns the set of possible next states.
- **Tree (Rooted Tree):** An undirected graph with a special node called the root, in which every node is connected to the root by exactly one path.
- **Truth Table:** A mathematical table used in logic to show the truth value of a compound proposition for all possible truth values of its constituent simple propositions.
- **Two-way Bounding:** A proof technique used to establish an exact result (often an equality) by proving both an upper bound and a lower bound for the quantity in question.
- **Uncountable Set:** An infinite set that cannot be put into a one-to-one correspondence with the set of natural numbers ($\mathbb{N}$).
- **Universal Quantifier ($\forall$):** "For all" or "for every". Denotes that every element in a set satisfies a given property.
- **Unrolling:** A technique for finding a closed-form expression for a recursive definition by repeatedly substituting the definition into itself until a pattern emerges.
- **Upper Bound:** A value that is greater than or equal to all values in a set. In optimization or proofs, a value that is known to be the maximum possible for a quantity.

- Vacuously True: A conditional statement ($P \rightarrow Q$) that is true because its hypothesis (P) is always false.
- Walk (in graph): A sequence of alternating vertices and edges, starting and ending with a vertex, where each edge connects the preceding and following vertices in the sequence.
- Wheel Graph (W_n): A graph formed by connecting a single central hub vertex to all vertices of a cycle graph C_n .
- Without Loss of Generality (WLOG): A common phrase in proofs indicating that a particular assumption is being made that does not affect the generality of the proof, often because other cases would be symmetrical or follow similar logic.